



Cloud And Data Universe



LangGraph

INTRODUCTION TO LANGGRAPH



Cloud And Data Universe

INTRODUCTION TO LANGGRAPH



If LangChain helps you build AI applications, LangGraph helps you build AI agents that can **think**, **remember**, **make decisions**, **use tools**, and **execute complex workflows** reliably.

THINK OF IT THIS WAY



LANGCHAIN

= Building blocks



LANGGRAPH

= Intelligent workflow engine



LangGraph is an **open-source** framework developed by the LangChain team for creating **stateful**, **multi-step AI agents**.



STATEFUL

Maintain context across steps



MULTI-STEP

Orchestrate complex workflows



TOOL INTEGRATION

Connect with APIs, databases, and more



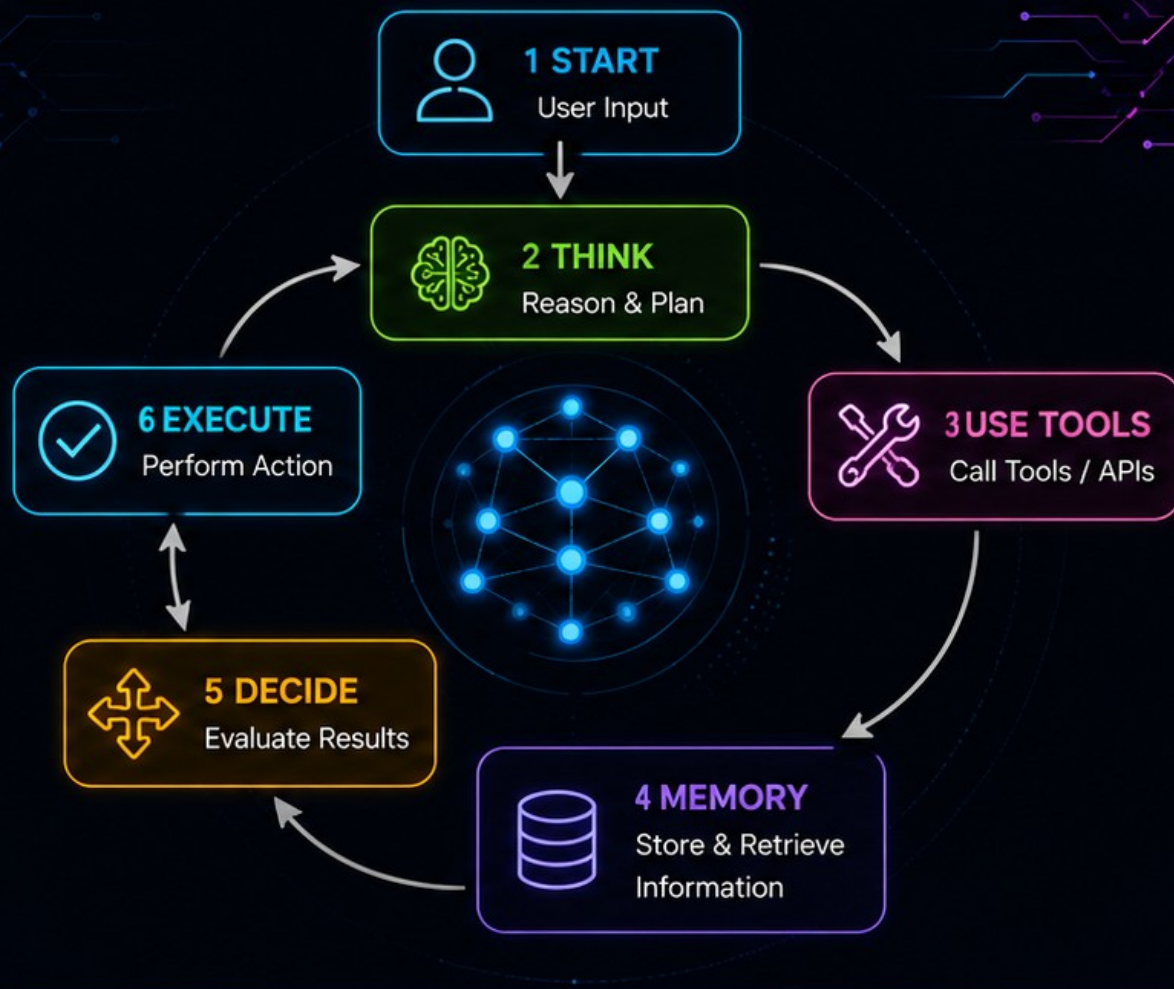
RELIABLE

Built for production, designed for scale

“

LangGraph turns your AI ideas into **intelligent**, **reliable**, and **scalable** agents.

HOW LANGGRAPH WORKS



WHY WAS LANGGRAPH CREATED?



LangGraph was created to build stateful, multi-step AI applications that can **reason, act**, and handle complex real-world workflows **reliably**.

EARLY LLM APPLICATIONS WERE SIMPLE.



EXAMPLE:

User:
What is Python programming?

LLM:
Python programming is...



Everything happens
in **one prompt**.



BUT REAL-WORLD AI APPLICATIONS ARE MUCH MORE COMPLICATED.

SUPPOSE THE USER ASKS:



Analyze this sales data,
query SQL Server,
generate Python code,
create charts,
explain insights,
and email me the report.



That is not one LLM call.
It involves **many steps**.



MANAGING
THESE
WORKFLOWS
BECAME
DIFFICULT
USING ONLY
CHAINS.



LANGGRAPH SOLVES THIS PROBLEM.



STATEFUL

Remembers
context across
steps



FLEXIBLE

Create complex
conditional
workflows



RELIABLE

Handles failures
and retries
gracefully



SCALABLE

Build agents
that can grow
with your needs

WHY IS IT CALLED "GRAPH"?



A graph is a **computer science** data structure.

It consists of:



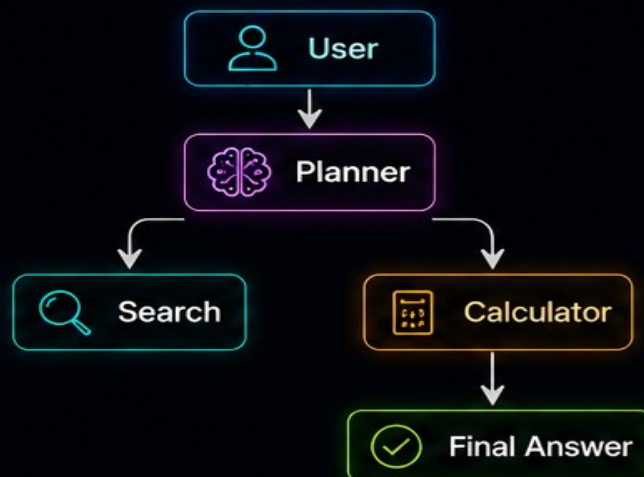
Nodes



Edges



Example



Each box
is a **node**.



Each arrow
is an **edge**.

NODES

A node performs one task.

Examples:



LLM Node



Tool Node



Python Node



SQL Node



Search Node



Memory Node

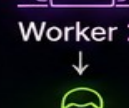
Think of each
node as a **worker**.



Worker 1



Worker 2



Worker 3



Each worker
performs **one job**.

EDGES

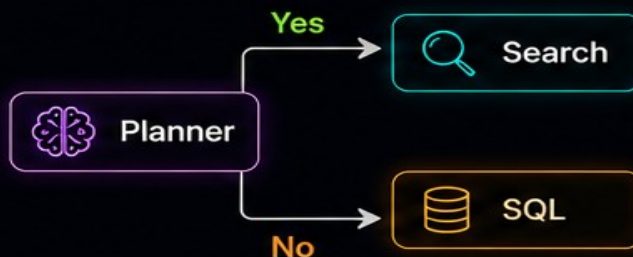
Edges decide **where to go next**.

1. Simple edge



One path
from **A**
to **B**.

2. Conditional edge



Different paths
based on
conditions.



The graph decides **dynamically**
based on the situation.

THE 8 PILLARS OF LANGGRAPH



1. STATE (THE BRAIN 🧠)



The State is the shared data that flows through the graph.

Every node:

- Reads the current state
- Performs its task
- Returns updates

LangGraph merges the updates automatically.

Example:

```
state = {
  "question": "...",
  "sql_query": None,
  "result": None,
  "answer": None
}
```

Without state, nodes cannot collaborate effectively.

2. NODES (THE WORKERS ⚙️)

Nodes perform the actual work.

A node can be:

- LLM
- Python function
- Tool
- SQL execution
- API call
- Human approval
- Another graph

Example



Think of each node as an employee with one responsibility.

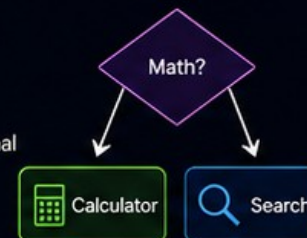
3. EDGES (THE DECISION MAKER ➡️)

Edges determine where execution goes next.

Types include:

- Normal edges
- Conditional edges
- Loops (using conditional routing)
- END transitions

Example



Edges make the workflow dynamic.

4. MEMORY & PERSISTENCE (THE MEMORY 💾)

LangGraph supports saving progress using checkpointing.

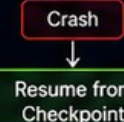
This enables:

- Conversation memory
- Long-running workflows
- Human approval
- Resume after failure
- Multi-session agents

Without persistence:



With persistence:



5. TOOLS (THE HANDS 🛠️)

LLMs can't directly interact with external systems. Tools give them capabilities.

Examples:

- SQL Database
- Python
- REST APIs
- Search engines
- Azure services
- Databricks
- Microsoft Fabric
- Power BI

Example



6. OBSERVABILITY (THE EYES 👁️)

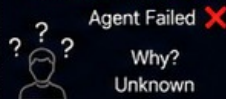
Observability lets you inspect everything happening inside your agent.

You can monitor:

- Node execution
- State changes
- Prompts
- Responses
- Tool calls
- Tokens
- Latency
- Errors
- Traces

Platforms such as LangSmith provide rich tracing and debugging capabilities.

Without observability:



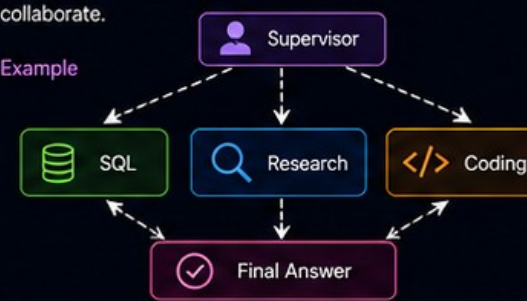
With observability:



7. MULTI-AGENT ORCHESTRATION (THE TEAM 👥)

Instead of one large agent, multiple specialized agents collaborate.

Example



Each agent focuses on what it does best.

8. HUMAN-IN-THE-LOOP (THE SUPERVISOR 👤)

Sometimes AI should pause for human approval.

Example



Common scenarios:

- Financial transactions
- Database updates
- Sending emails
- Healthcare
- Legal documents



These 8 pillars work together to help you build **powerful, reliable, scalable,** and **production-ready** AI agents and workflows.



Smarter Agents



Better Collaboration



Deep Visibility



Reliable Execution



Lower Costs



Production Ready

STATE: THE HEART OF LANGGRAPH



The most important concept is **State**.

WITHOUT STATE



Node A
doesn't know what
Node B
did.

STATE IS SHARED MEMORY.

EXAMPLE

```
State
{
  user_question,
  sql_query,
  search_results,
  python_code,
  final_answer
}
```



Every node
can **read** it.



Every node
can **update** it.

IMAGINE A SHARED WHITEBOARD.

Node A
writes



Node B
reads



Node C
updates



Everyone collaborates through
the **same shared state**.

WHY STATE IS CRUCIAL



Maintains **context**
across steps.



Enables complex
decision making.



Allows nodes to
build on each other's
outputs.



Provides **consistency**
and **reliability**.



Supports **loops**,
retries, and **dynamic**
workflows.



Powerful foundation
for **intelligent**
AI agents.

This example is very close to how **LangGraph** is actually used.

**USER ASKS:**

"Review my **resume**
and suggest **improvements**."

**INITIAL STATE**

```
state = {
  "resume": "...",
  "job_description": "...",
  "skills": None,
  "ats_score": None,
  "suggestions": None
}
```

WORKFLOW OVERVIEW

1



EXTRACT SKILLS

2

CALCULATE
ATS SCORE

3

GENERATE
SUGGESTIONSFINAL
STATE**1 NODE 1: EXTRACT SKILLS**

- Reads the resume.

Updates

```
{
  "skills": [
    "Python",
    "Azure",
    "SQL"
  ]
}
```

STATE BECOMES

```
{
  "resume": "...",
  "job_description": "...",
  "skills": [
    "Python",
    "Azure",
    "SQL"
  ],
  "ats_score": None,
  "suggestions": None
}
```



This node extracts the important skills
from the resume and stores them in state.

2 NODE 2: CALCULATE ATS SCORE

- Reads
- skills
 - resume
 - job_description

Updates

```
{
  "ats_score": 82
}
```

STATE BECOMES

```
{
  "resume": "...",
  "job_description": "...",
  "skills": [
    "Python",
    "Azure",
    "SQL"
  ],
  "ats_score": 82,
  "suggestions": None
}
```



This node calculates the ATS score based on
overlap between skills, resume and job description.

3 NODE 3: GENERATE SUGGESTIONS

- Reads
- Resume
 - Skills
 - ATS Score
 - Job Description

Updates

```
{
  "suggestions": [
    "Add Azure DevOps experience",
    "Include measurable achievements",
    "Mention Databricks projects"
  ]
}
```

FINAL STATE

```
{
  "resume": "...",
  "job_description": "...",
  "skills": [
    "Python",
    "Azure",
    "SQL"
  ],
  "ats_score": 82,
  "suggestions": [
    "Add Azure DevOps experience",
    "Include measurable achievements",
    "Mention Databricks projects"
  ]
}
```



All nodes have updated the shared state.
This is the final result.

THE KEY IDEA

Think of the **state** as
a single **shared folder**
that travels through
the entire workflow.

**EVERY NODE:**

Reads the current state.



Performs **one specific task**.



Returns **only the fields** it
wants to update.



LangGraph automatically merges
those updates into the shared state
before passing it to the next node.

This **shared, evolving state** is what
allows independent nodes to **collaborate**
without directly calling or depending
on each other.

HOW LANGGRAPH EXECUTES

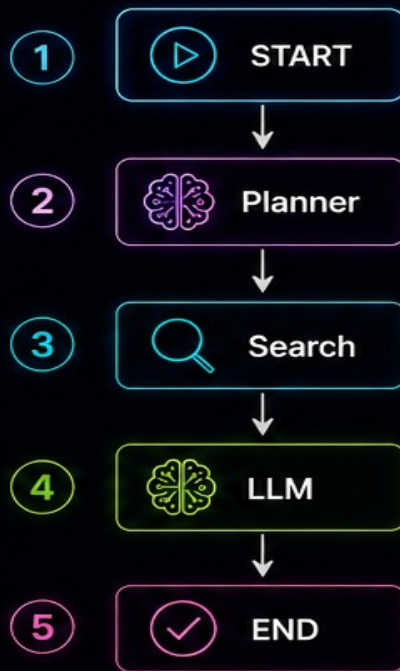


LangGraph executes nodes **step by step**, passing a **shared state** along the way.

1. IMAGINE THE GRAPH



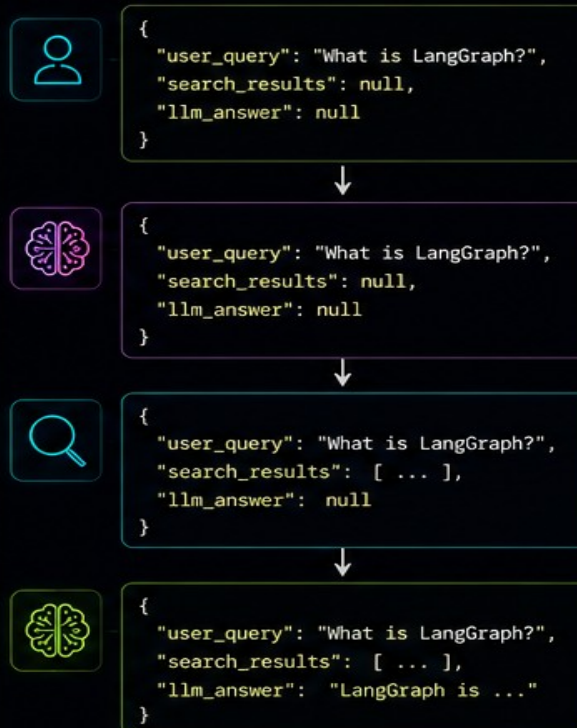
2. EXECUTION BEGINS



The graph executes nodes **one after another**.

3. STATE FLOWS THROUGH

Every node receives the **current state**.



The **shared state** is passed and **evolves** at every step.

4. HOW EACH STEP WORKS

EXAMPLE

Current State



Node Modifies It

```

return {
  "search_results": results
}
  
```

Graph Merges the Update

```

{
  "user_query": "What is LangGraph?",
  "search_results": [ ... ],
  "llm_answer": null
}
  
```

Next Node Sees the New State

```

{
  "user_query": "What is LangGraph?",
  "search_results": [ ... ],
  "llm_answer": null
}
  
```



Each node does **one job**, returns only updates, and the graph **handles** the rest.



KEY TAKEAWAY

LangGraph executes nodes in order, passing a **shared state**.
Nodes focus on their task, while the graph manages the **flow** and **state**.



CONDITIONAL ROUTING



One of LangGraph's greatest strengths is **decision-making**.

EXAMPLE



User asks
"Calculate 10×20 "



Planner decides
to use "Calculator"

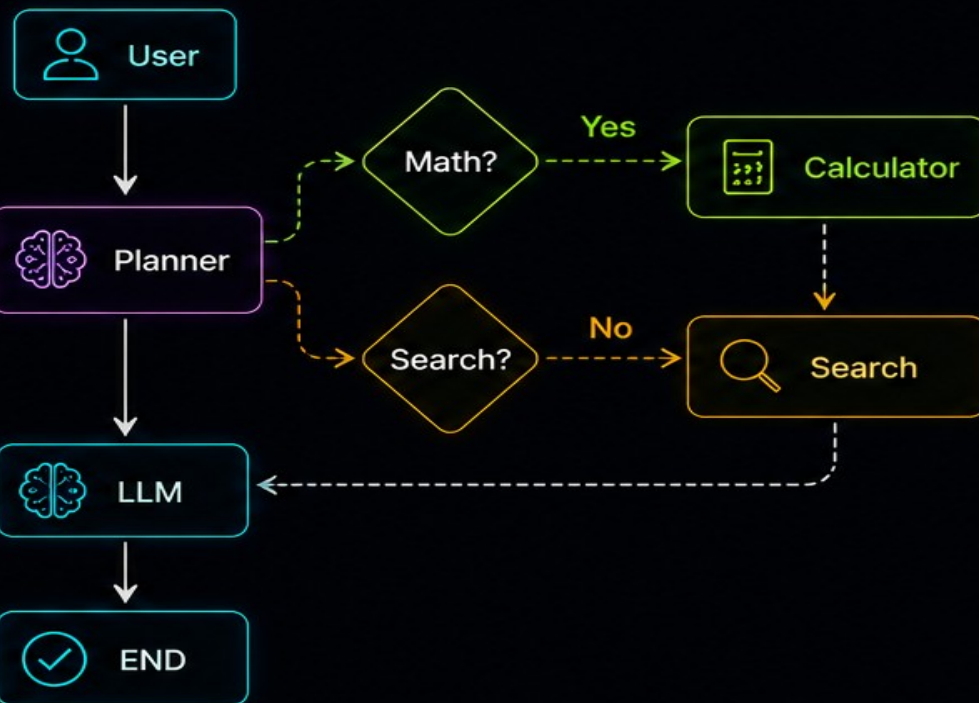


Calculator
instead of "Search"



Answer returned
200

GRAPH



PSEUDO LOGIC

```
if question contains numbers:
    route = "Calculator"
else:
    route = "Search"
```



Planner decides the
best next step based
on the question.



The graph takes
the **appropriate path**.



THE POWER OF ROUTING



Makes the system
adaptive.



Executes only the
necessary steps.



Saves time, resources,
and **tokens**.



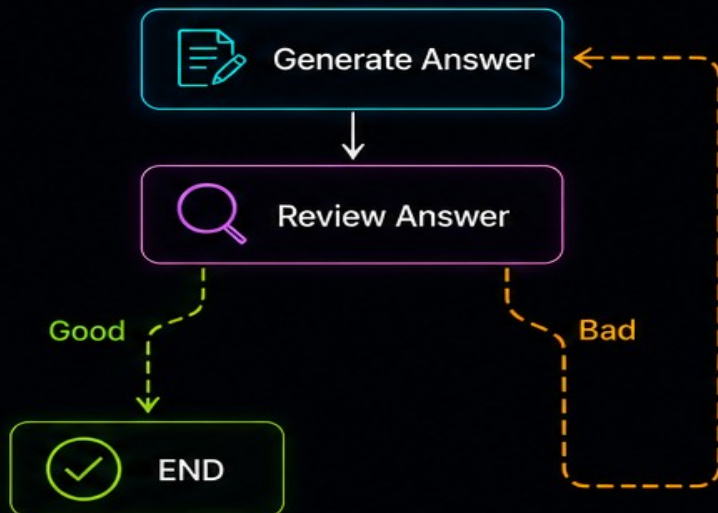
Conditional routing allows LangGraph to make **smart decisions** and build truly **intelligent workflows**.

LOOPS (CREATED USING EDGES)



LangGraph doesn't have a separate "loop edge." Instead, you create loops by **connecting edges** back to an **earlier node**.

LOOP EXAMPLE



CODE EXAMPLE

```
builder.add_conditional_edges(
    "review",
    review_answer,
    {
        "retry": "generate",
        "finish": END
    }
)
```



This allows **iterative improvement** until a stopping condition is met.

HOW IT WORKS



1. Start at "Generate Answer"
The LLM generates a response.



2. Review the Answer
A node reviews the response.



3. Decide the Path
If the answer is **good** → END
If **bad** → go back and retry



4. Repeat Until Good
The loop continues until the stopping condition is satisfied.

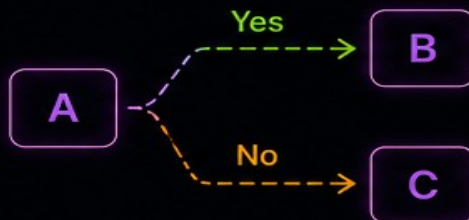
VISUAL SUMMARY

1 NORMAL EDGE



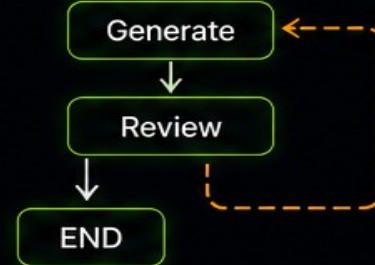
A **direct flow** from one node to the next.

2 CONDITIONAL EDGE



The next node **depends on a condition**.

3 LOOP



An edge that goes back to an earlier node to **repeat** the process.

WHY LOOPS ARE POWERFUL



Enable **iterative** refinement for better results.



Allow the agent to **self-correct** and improve outputs.



Continue until a **stopping** condition is met.



Build more **intelligent**, reliable AI workflows.

TOOL CALLING



Agents frequently need **external tools** to get the job done.

WHAT IS TOOL CALLING?



The agent can request and use **external** systems, APIs, databases, or applications to complete the task.

EXAMPLES OF TOOLS



SQL



Python



REST APIs



Weather APIs



Azure



Databricks

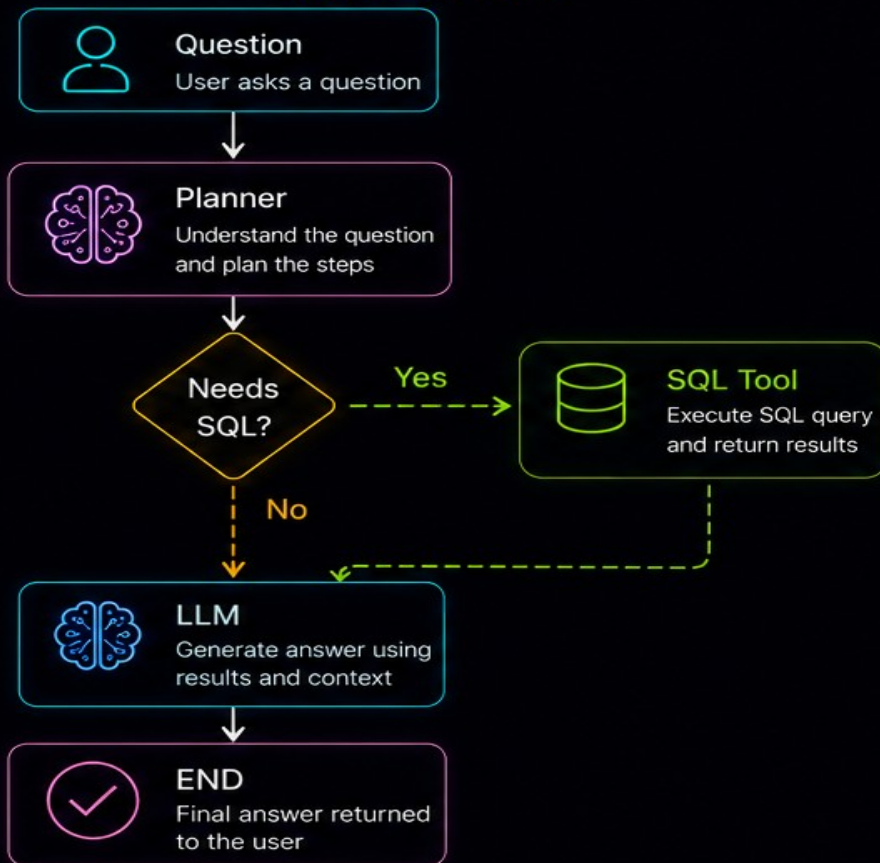


Power BI



Search Engines

GRAPH EXAMPLE



HOW IT WORKS



- 1 User asks a question.
The graph starts with the user's input.



- 2 Planner analyzes the question.
It decides the best next step.



- 3 Graph checks if a tool is needed.
Based on the plan and context.



- 4 Tool is called if required.
Example: SQL query is executed.



- 5 Results go back to the LLM.
LLM uses them to generate a final answer.



- 6 Answer is returned.
The task is complete.



The graph determines **when** and **which tools** are required.



WHY TOOL CALLING MATTERS



Extends agent capabilities beyond the LLM's knowledge.



Connects to **real systems** and live data.



Improves **accuracy** with authoritative information.



Enables **automation** of complex, real-world tasks.

MEMORY



Traditional LLMs forget after each call.
LangGraph can preserve memory through its **state** and **checkpointing** mechanisms.

WHAT IS MEMORY?



Memory is information from past interactions, decisions, and results that the agent can access later.

WHERE MEMORY LIVES IN LANGGRAPH



STATE

Holds the current conversation and all relevant information.



CHECKPOINTING

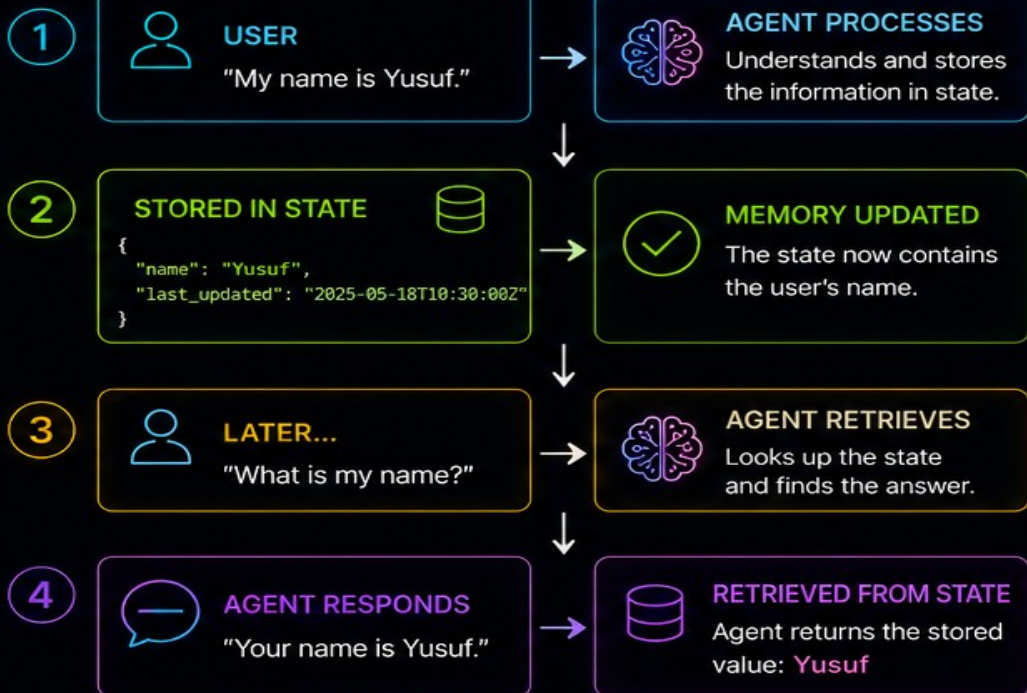
Saves the state so memory can persist across sessions, crashes, or restarts.



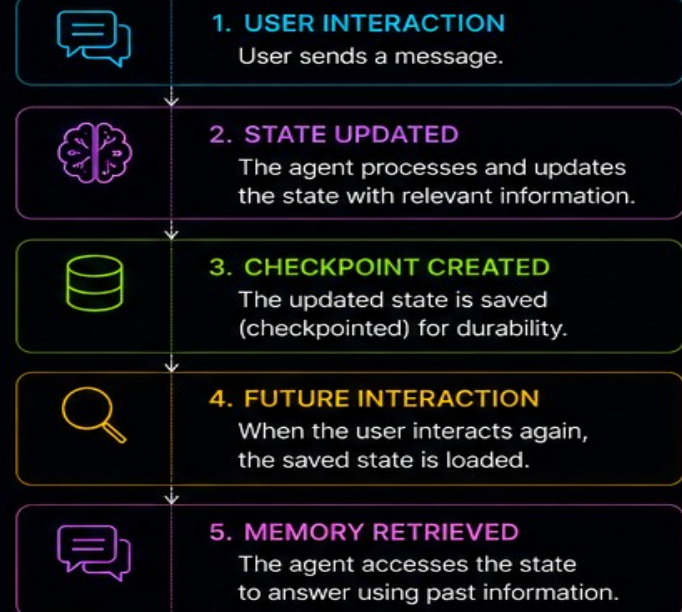
KEY IDEA

State is the working memory.
Checkpointing is long-term memory.

EXAMPLE CONVERSATION



HOW MEMORY WORKS IN LANGGRAPH



PERSISTENCE OPTIONS



BENEFITS OF MEMORY



Enables natural, **context-aware** conversations.



Improves **accuracy** by using past information.



Saves **time** by avoiding repeated questions.



Supports **long-running** workflows and user sessions.

IMPORTANT NOTE



Memory persists across multiple interactions when configured.
You control **what to store**, **how long** to keep it, and when to **forget** it.



LangGraph turns stateless LLMs into **stateful**, **memory-aware** agents that can **remember**, **reason**, and deliver **better experiences**.

MULTI-AGENT SYSTEMS IN LANGGRAPH

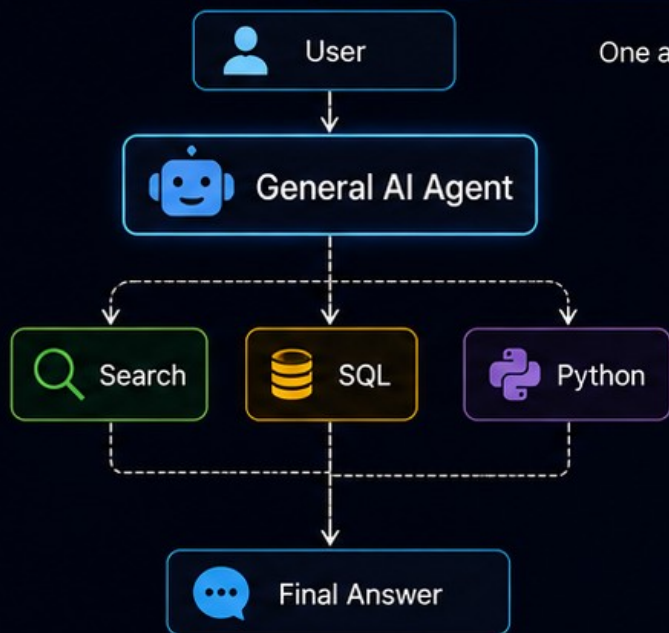


One of the most powerful capabilities of LangGraph is its support for Multi-Agent Systems. Instead of asking one AI agent to perform every task, you create **multiple specialized AI agents**, where each agent is responsible for **one specific job**.



Think of it as building an **AI team** instead of hiring one person to do everything.

SINGLE AGENT vs MULTI-AGENT



One agent performs every task.

The same agent must:

- ✓ Understand the question
- ✓ Decide which tool to use
- ✓ Generate SQL
- ✓ Write Python
- ✓ Analyze results
- ✓ Produce the final response



As tasks become more complex, this agent becomes harder to manage.



MULTI-AGENT SYSTEM

Instead, split responsibilities.



Each agent has a single responsibility.

- ✓ Better accuracy
- ✓ Easier to manage
- ✓ Scalable
- ✓ More reliable
- ✓ Easier to debug
- ✓ Modular and reusable



A team of specialists always outperforms a generalist.



Multi-Agent Systems in LangGraph allow you to build scalable, maintainable, and **production-ready AI applications** by leveraging the power of **collaboration**.



Focused Agents



Better Collaboration



Higher Accuracy



Modular Design



Production Ready

THINK OF A COMPANY



Imagine you own a
software company.

Would one employee do everything?



Sales



Finance



Development



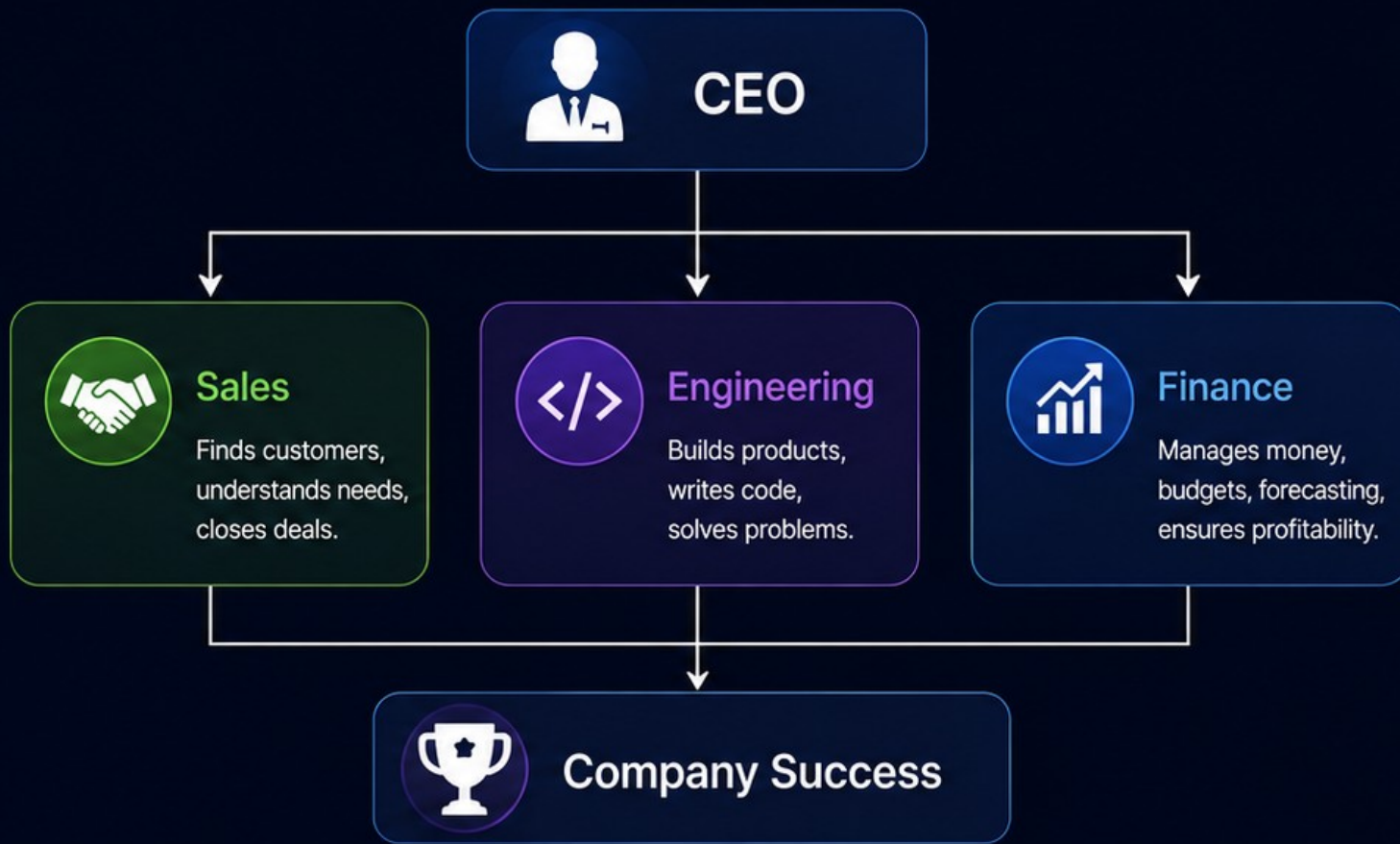
HR



Marketing



Of course not.
Instead you hire **specialists.**



A **Multi-Agent System** works
exactly the same way.



Specialized
Agents



Focused
Responsibilities



Better
Collaboration



Higher
Efficiency



Better
Results

In LangGraph, an agent is often just **another node** or even **another graph**.

This means you can:



Call one agent from another.



Route between specialized agents using conditional edges.



Share state across agents.



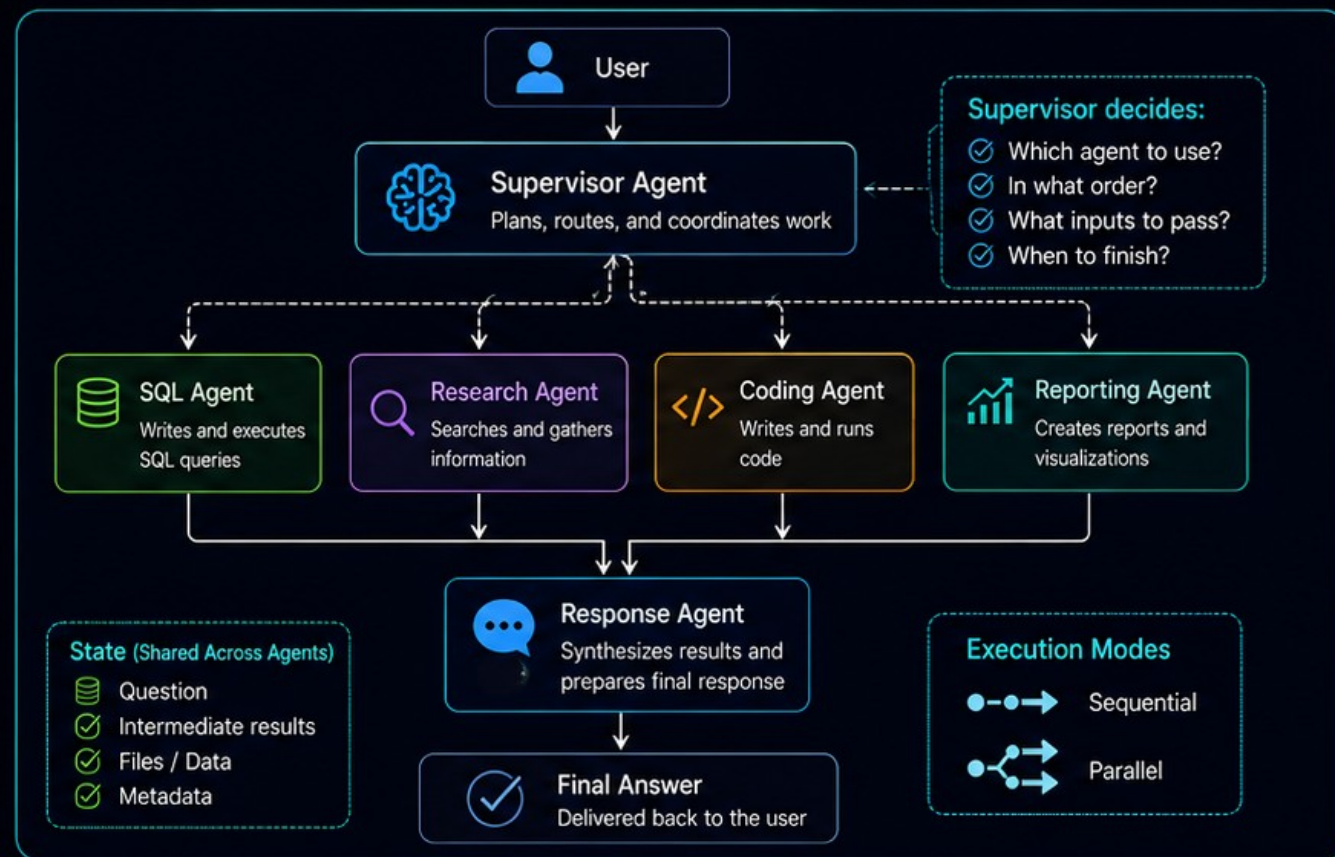
Run agents sequentially or in parallel.



Allow a supervisor agent to orchestrate the workflow.



Because agents are built as composable graphs, you can **nest** and **reuse** them just like any other LangGraph component.



KEY TAKEAWAYS



A Multi-Agent System is a team of specialized AI agents working together toward a common goal.



Each agent focuses on a single responsibility, such as SQL, research, coding, or reporting.



A Supervisor Agent coordinates the workflow by deciding which agent should execute and when.



Multi-agent systems are more modular, scalable, maintainable, and accurate than relying on a single general-purpose agent for complex tasks.



LangGraph makes multi-agent architectures natural by treating agents as reusable nodes or subgraphs connected through shared state and intelligent routing.

HUMAN-IN-THE-LOOP



Sometimes the workflow **pauses** for **human approval**.

WHY IT MATTERS



Safety

Prevents risky or unintended actions.



Accuracy

Humans catch mistakes or edge cases.



Compliance

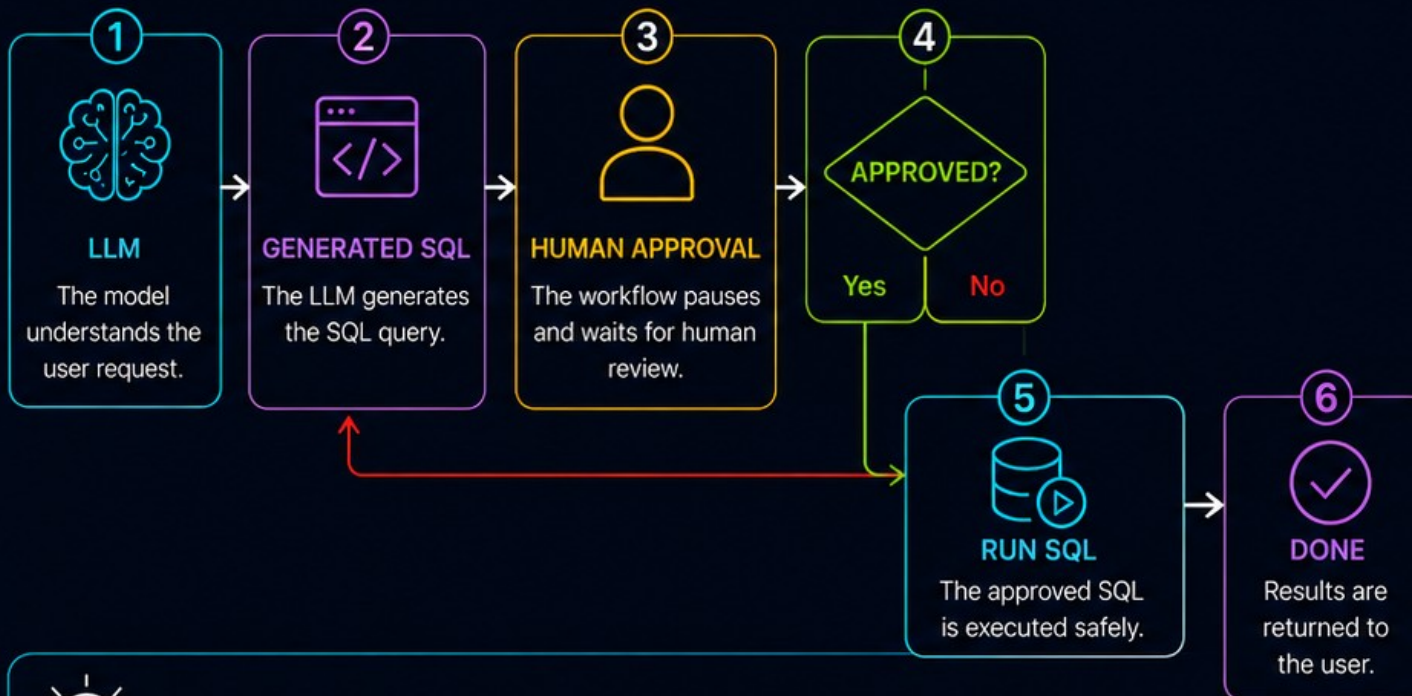
Helps meet policies and regulations.



Trust

Human oversight builds confidence in the system.

WORKFLOW EXAMPLE



This is valuable for production systems where actions like **database updates**, **sending emails**, **payments**, or **deleting data** require confirmation.



HOW IT WORKS IN LANGGRAPH



1 Workflow reaches a human approval node.



2 The graph pauses and waits for human input.



3 Human reviews and approves or rejects.



4 The graph continues based on the human decision.



5 If approved, the action is executed.



6 The result is returned to the user.

COMMON USE CASES



Database updates



Sending emails



Payments and refunds



Deleting records



Deploying to production



Any critical operation

CHECKPOINTING



LangGraph can save progress. If execution stops, it can later **resume** from the **last saved checkpoint** instead of starting over.

WHAT IS CHECKPOINTING?



Checkpointing means saving the **current state** of the graph at certain points during execution.

WHY IT MATTERS



Fault Tolerance

Recover from crashes or interruptions.



Time Saving

Resume from where you left off, not from the start.



Cost Efficient

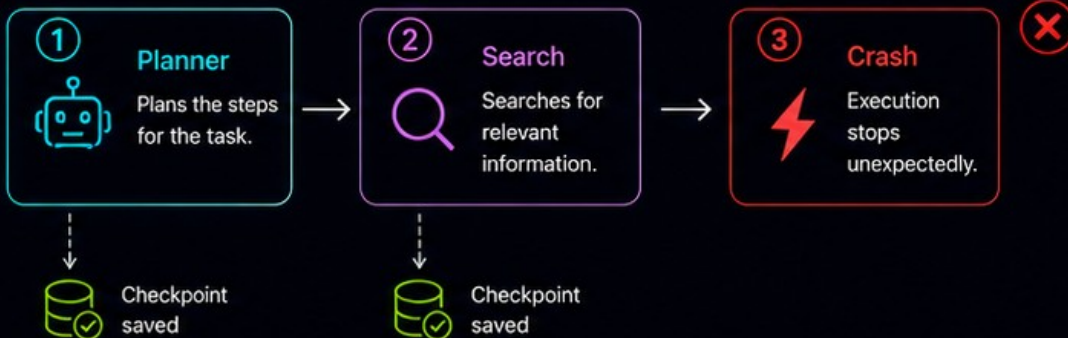
Avoid re-running expensive operations.



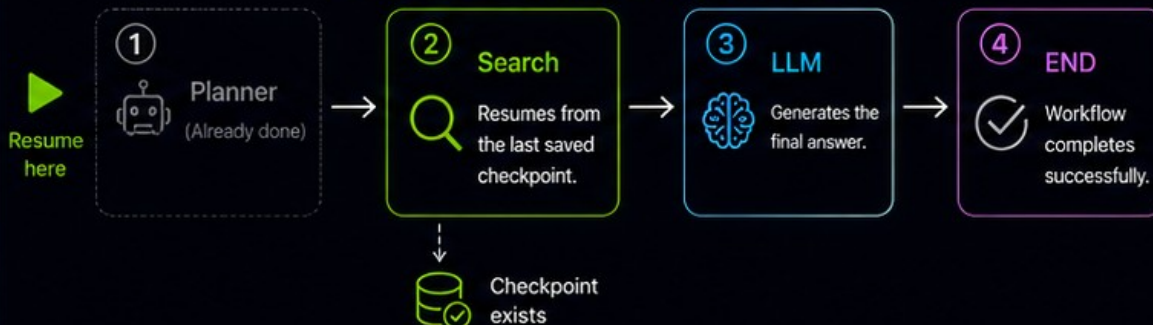
Reliability

Essential for production workflows.

EXAMPLE: EXECUTION STOPS (CRASH)



RESUME FROM LAST CHECKPOINT



HOW CHECKPOINTING WORKS



State Capture

LangGraph captures the current state after each step (or at defined intervals).



State Storage

The state is saved to a checkpoint store (e.g., database, file, Redis).



Failure Occurs

If the execution stops or crashes, the last checkpoint remains safe.



Resume Execution

LangGraph loads the last saved state and continues from there.

WHERE IT HELPS



Long-running workflows



Multi-step agents



External tool operations



Human-in-the-loop flows



ETL / Data pipelines



Checkpointing makes your LangGraph workflows **resilient**, **efficient**, and **production-ready**.



Save State
(Checkpoint)



Crash / Stop
(Unexpected)



Resume from
Last Checkpoint



Complete
Successfully

WHAT IS OBSERVABILITY?

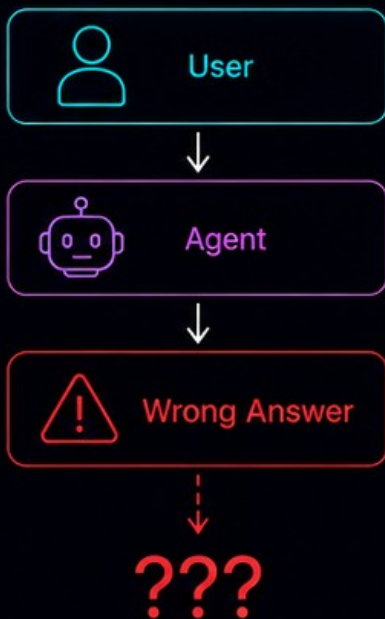


Observability is the ability to **see**, **inspect**, **debug**, **monitor**, and **understand** everything happening inside your AI agent while it executes.



Think of it as a **flight recorder** for your AI agent.

WITHOUT OBSERVABILITY



YOU HAVE NO IDEA:



Which node executed



Which tool failed



What the LLM thought



Which prompt was sent



How much it cost



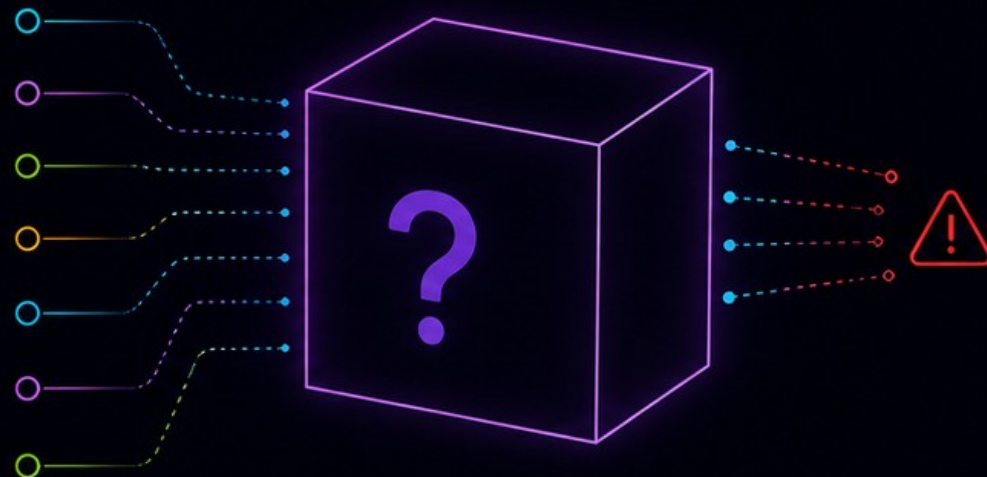
Why it took 40 seconds



Which state variable became incorrect



EVERYTHING IS A BLACK BOX.



WHY OBSERVABILITY MATTERS



Faster
Debugging
Find issues
quickly



Improved
Reliability
Catch failures
early



Performance
Insights
Optimize
execution



Cost
Visibility
Track usage
and spend



Better
Decisions
Understand
behavior



Build Trust
Increase
confidence



If you can't **observe** your agent, you can't **improve** your agent.

WHY IS OBSERVABILITY IMPORTANT?



Observability gives you **complete visibility** into your AI agent's execution, so you can **debug, fix, and improve** with confidence.



Imagine your
Azure Data Engineering agent.



User asks:

"Load customer data from
Azure SQL into Fabric."

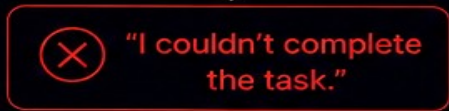


Instead of succeeding,
it replies:

"I couldn't complete
the task."



WITHOUT OBSERVABILITY

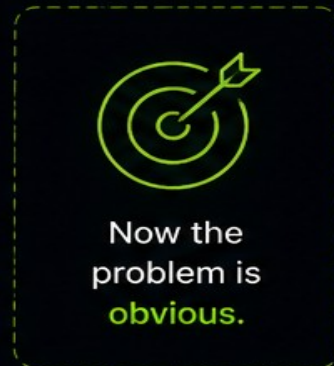
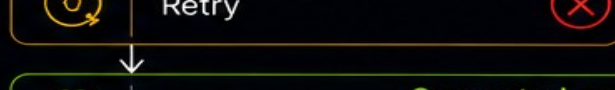
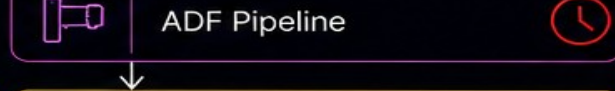
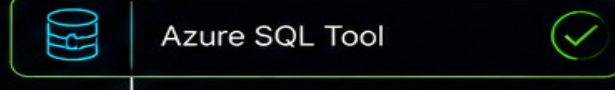
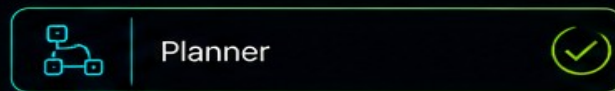


WHY?

No idea.



WITH OBSERVABILITY



OBSERVABILITY HELPS YOU:



Find issues
faster

Debug quickly
and efficiently.



Improve
reliability

Catch failures
before users do.



Optimize
performance

Reduce latency
and bottlenecks.



Control costs

Track token usage,
tool calls, and
resource spend.



Build trust

Understand behavior
and make better
decisions.

WHAT CAN WE OBSERVE?



A good observability platform lets you **inspect everything** inside your AI agent's execution.

1 NODE EXECUTION

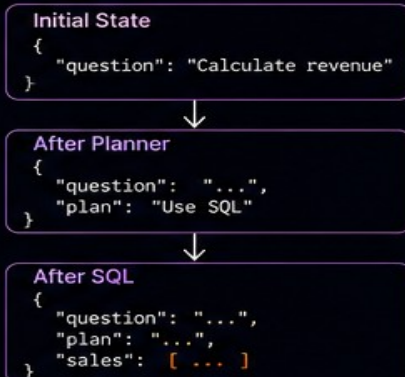


- Which nodes executed?
- How long did each take?

Example

Planner	0.8s
Search	1.2s
Python	3.6s

2 STATE CHANGES



You can watch the state evolve step by step.



3 LLM PROMPTS

Exactly what prompt was sent.

System:
You are a SQL expert.

User:
Calculate monthly revenue.

Context:
...



This is invaluable for prompt debugging.

4 LLM RESPONSES

Observe the raw model output before downstream processing.

Thought: Need SQL.

Action: Run Query

Observation: Retrieved 12 rows.

...



Understand the model's reasoning and intermediate outputs.

5 TOOL CALLS

Every tool invocation can be traced.



or



You can inspect inputs, outputs, latency, and failures.

6 TOKEN USAGE

Example

Planner
Input: 500 tokens
Output: 150 tokens
Cost: \$0.002

Across the whole graph:



Total:
5,600 tokens
\$0.09

This helps control operational costs.

7 EXECUTION TIME



You can identify bottlenecks.

8 ERRORS

Instead of "Something failed."

You see



Full visibility helps you resolve issues fast.



**OBSERVABILITY TURNS
A BLACK BOX INTO
A GLASS BOX.**



See what happens



Debug faster



Improve reliability



Control costs



Optimize performance

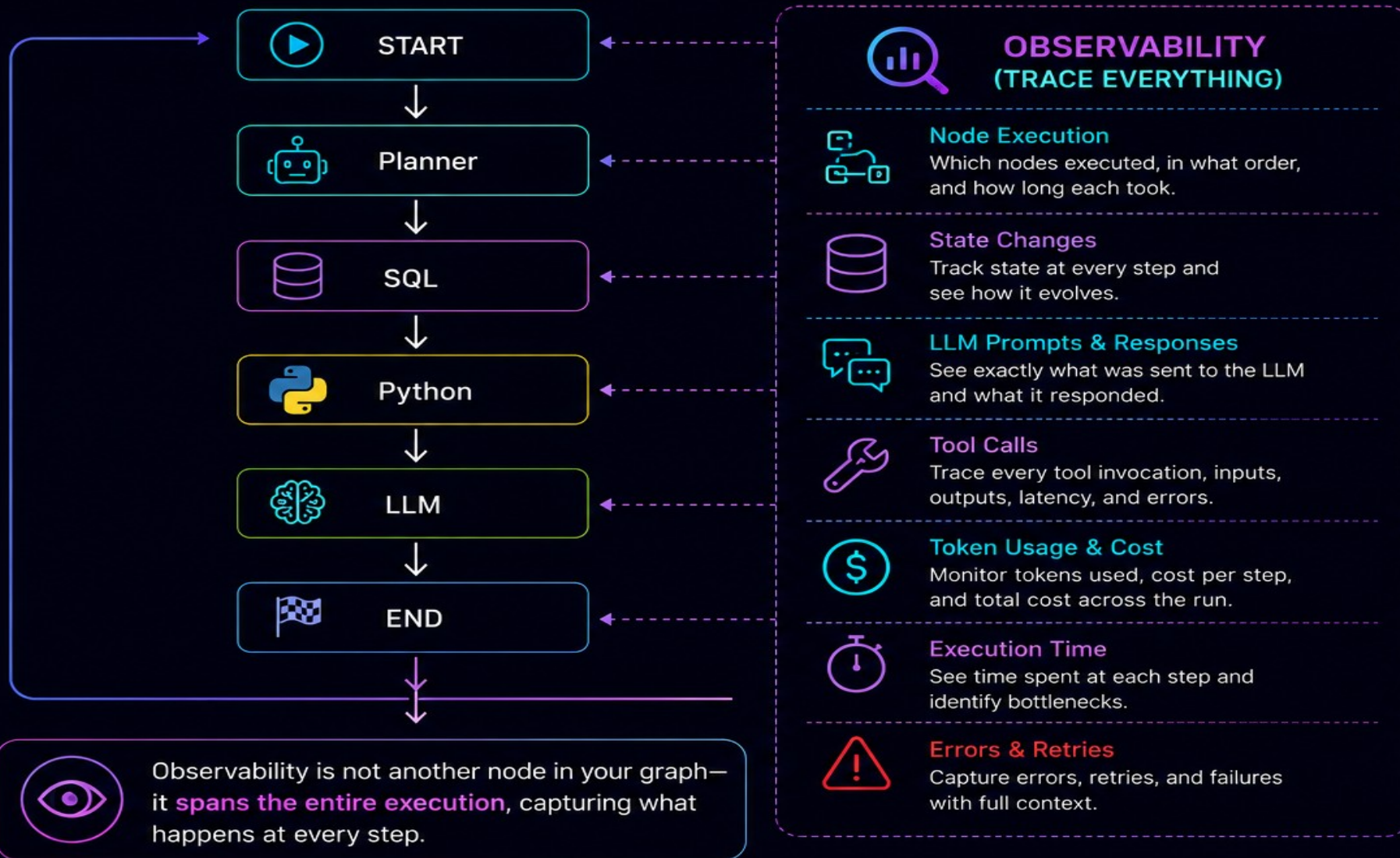


Ship with confidence

WHERE DOES OBSERVABILITY FIT?



Observability spans your entire Langeraph execution, capturing what happens at every step.



OBSERVABILITY VS LOGGING



Many newcomers think they're the same,
but they serve **different purposes**.

 LOGGING		 OBSERVABILITY
 Writes messages to log files	VS	Provides end-to-end execution traces 
 Text-based	VS	Structured, visual traces 
 Difficult to follow across many components	VS	Shows complete workflow execution 
 Manual debugging	VS	Interactive inspection and analysis 
 Limited context	VS	Includes state, prompts, tool calls, timing, and costs 

 LOGGING Tells you what happened at a single location .	VS	OBSERVABILITY Tells you what happened, why it happened, and how everything is connected . 
---	-----------	--



Logging is a flashlight. **Observability** is a control room. One shows you a small part. The other shows you the **whole picture**.



The LangChain ecosystem provides **LangSmith**, which is the **primary observability platform** for LangChain and LangGraph applications.

WITH LANGSMITH YOU CAN:



Trace every graph execution



Visualize node-by-node execution



Inspect prompts and responses



Compare runs



Replay executions



Track latency and token usage



Monitor failures



Evaluate agent quality over time



Collaborate with your team on debugging and improvements

Projects > Sales Copilot Agent > Run 1f2e3d4c

Compare

Share

Add to Dataset

Run

Timeline

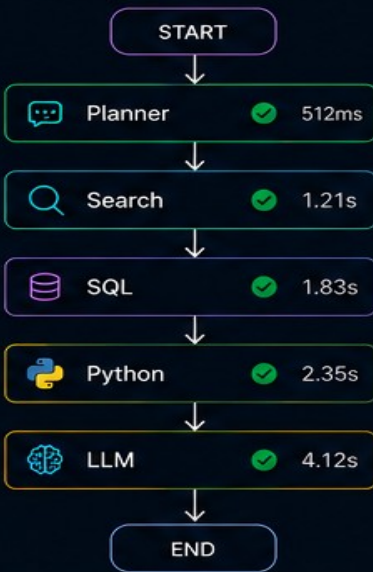
Tree

Traces

Observations

Metadata

Graph Execution



Run Details

Run ID	1f2e3d4c-9b8a-4c7d-a6e5-9b8f0a1b2c3d
Start Time	May 24, 2024 10:21:15 AM
Total Time	10.31s
Status	✔ Success
User	alice@example.com
Input	How was revenue this month?
Output	Revenue increased by 12% compared to last month.

Summary

Tokens	Total Cost	
5,742	\$0.0942	
(2,134 in / 3,608 out)		
Total Time	Nodes	Errors
10.31s	6	0

Prompt

System
You are a sales data analyst...

User
How was revenue this month?

Context
{...}

Show more

Response

To answer your question, I queried the sales data for this month and compared it with last month.

Revenue increased by 12% compared to last month.

...

Show more

Metadata

Model	gpt-4o
Temperature	0.2
Top P	0.9
Tags	sales, dashboard, revenue



Think of it as the equivalent of **application performance monitoring (APM)** tools like **Application Insights** or **Datadog**, but designed specifically for **LLM applications**.



Application
Insights



DATADOG



LangSmith

For LLM Applications

LANGCHAIN VS. LANGGRAPH



LangChain is great for getting started.
LangGraph is built for complex, **stateful**, **production-ready** AI applications.



 FEATURE	 LANGCHAIN	 LANGGRAPH
 Simple prompt chains	✓	✓
 Tool calling	✓	✓
 Prompt templates	✓	✓
 Memory	Basic	Advanced state management
 Branching logic	Limited	Rich conditional routing
 Loops	Limited	Native support
 Stateful execution	Limited	Core design
 Checkpointing	Limited	Native support
 Multi-agent orchestration	Possible	First-class support
 Production AI workflows	Good	Excellent



KEY TAKEAWAY

Choose LangChain when:

- Your workflow is simple
- You need quick implementation
- You don't need complex control flow

Choose LangGraph when:

- Your workflow is complex or stateful
- You need branching, loops, and retries
- You're building production-grade AI systems



A practical order for learning LangGraph is:

1

STATE

(The Brain 🧠)



Understand how information flows through the graph.

Example

```
{
  "question": "...",
  "sql_query": null,
  "result": null,
  "answer": null
}
```

2

NODES

(The Workers ⚙️)



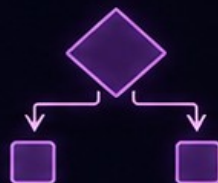
Learn how work is performed by different components.

Example

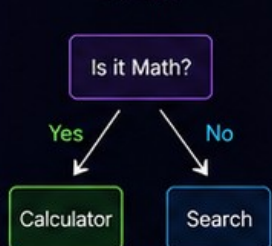
3

EDGES

(The Decision Maker ⚡)



Control execution paths using connections and conditions.

Example

4

TOOLS

(The Hands 🛠️)



Connect to external systems and give capabilities to your agents.

Example

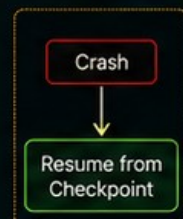
5

MEMORY & PERSISTENCE

(The Memory 📁)



Build long-running agents with memory, checkpoints and resume capability.

Example

6

OBSERVABILITY

(The Eyes 👁️)



Debug, monitor and optimize your graphs with full visibility and traces.

Example

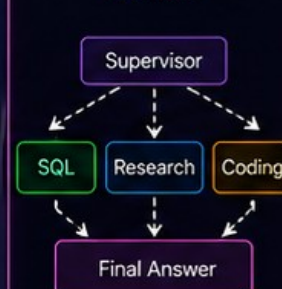
7

MULTI-AGENT SYSTEMS

(The Team 👥)



Coordinate multiple specialized agents to solve complex problems.

Example

8

HUMAN-IN-THE-LOOP

(The Supervisor ✅)



Add human oversight and approval in critical steps.

Example

Follow this roadmap **step by step** to go from beginner to building **production-ready** AI agents with **LangGraph**.



Structured Learning



Practical Progression



Real-World Applications



Production Ready



STREAMLIT



WHAT IS STREAMLIT?



Streamlit is a Python library that allows you to build web applications **using only Python**.

TRADITIONAL WEB DEVELOPMENT

HTML



HTML

CSS



CSS

JavaScript



JavaScript

✗ Requires multiple technologies

WITH STREAMLIT



PYTHON ONLY

No HTML, CSS or JavaScript required!



Build powerful apps with only Python

EXAMPLE

```
1 import streamlit as st
2
3 st.title("My First App")
4 st.write("Hello World")
```



OUTPUT



My First App

Hello World



Streamlit **automatically** creates a beautiful web page from your Python code.



KEY TAKEAWAY:



WRITE PYTHON



GET A WEB APPLICATION



THINK OF STREAMLIT AS A TRANSLATOR



Streamlit is the **translator** between your Python code and the web interface your users see.

YOU WRITE (PYTHON)

```
st.button("Submit")
```



STREAMLIT TRANSLATES INTO (WEB BROWSER)

Submit

Streamlit translates it into a **button** inside a web browser.

YOU WRITE (PYTHON)

```
st.text_input("Enter Name")
```



STREAMLIT TRANSLATES INTO (WEB BROWSER)

Enter Name

Streamlit translates it into a **text input box** inside a web browser.



YOU NEVER WRITE HTML.

You just write Python. Streamlit handles the rest!



TRADITIONAL WEB DEVELOPMENT



HTML
Page Structure



CSS
Styling



JavaScript
Interactivity



Python
Backend



Lots of **different languages**.

WITH STREAMLIT



Python



Streamlit



Beautiful Web Application



Only **one language**.

WHY WAS STREAMLIT CREATED?



Imagine you're
a **Data Scientist**.



You built an
amazing **AI model**.



Now you want
people to use it.



Without Streamlit,
you would need to learn
web development.

With **Streamlit**, you simply do

`st.file_uploader()`



`st.button()`



`st.image()`



`st.chat_input()`



and your
application
is **ready!**



**STREAMLIT LETS DATA PEOPLE FOCUS ON WHAT MATTERS:
DATA, MODELS, AND IMPACT – NOT FRONTEND COMPLEXITY!**





WHAT CAN WE BUILD?

Using **Streamlit** you can build:



Dashboards



AI Chatbots



Data Analysis Apps



Machine Learning Applications



Image Processing Apps



PDF Summarizers



SQL Query Tools



RAG Applications



For your course, you'll build an **AI chatbot**.



HOW DOES STREAMLIT WORK?

This is the **most important concept**.

Every time the **user interacts** with the page,



clicks
a button



enters
text

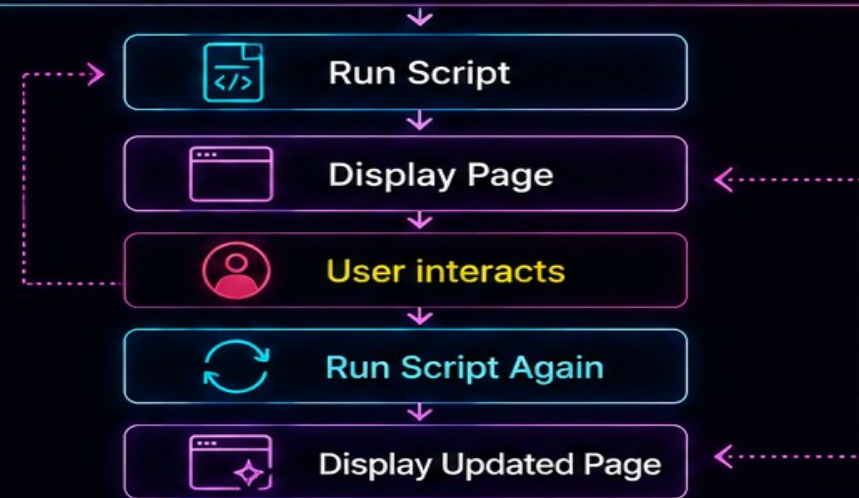


moves
a slider



sends a
chat message

Streamlit runs the **entire Python script** again.



This is called **rerunning the script**.



Streamlit automatically **reruns** your Python script on every interaction and **updates the page** to reflect the changes.





THEN WHY DOESN'T EVERYTHING DISAPPEAR?

Because **Streamlit** provides **Session State**.



User types



Script reruns



Session State
remembers previous values



Page is rebuilt



Without **Session State**,
every rerun would start from scratch.



A TYPICAL STREAMLIT PROGRAM

Almost every **Streamlit** application follows this pattern.



Import Libraries



Configure Page



Create Widgets



Read User Input



Process Data



Display Output



FOR EXAMPLE



```
1 import streamlit as st
2 st.set_page_config(page_title="Calculator")
3 st.title("Calculator")
4 number = st.number_input("Enter Number")
5 st.write(number * 2)
```

THE FLOW IS:



Open App



Show Title



Show Number Input



Wait for User



User Enters Value



Run Script Again



Display Result



IN OUR CHATBOT

Your chatbot follows the same pattern.



User Types
Question



`st.chat_input()`



Send Question
to LangGraph



LangGraph
calls LLM



LLM generates
response



Display response
using `st.chat_message()`



Streamlit **reruns** your script on every interaction,
but **Session State** keeps your app and data consistent.

